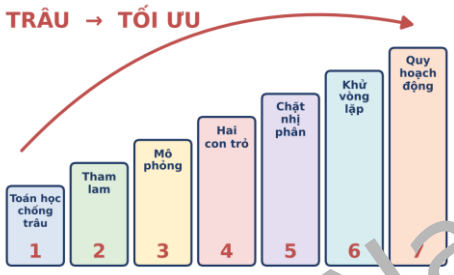
Tổng hợp và biên soạn: Lê Thành Nam

TÀI LIỆU BỒI DƯỠNG HỌC SINH GIỎI TIN HỌC CẤP TỈNH

TỪ TRÂU ĐẾN TỐI ƯU

Bảy con đường tư duy thuật toán



Lê Thành Nam

Chương 1. TOÁN HỌC VÀ CÁCH CHỐNG TRÂU
Chương 2. SỐ HỌC, XÂU VÀ SẮP XẾP
Chương 3. THAM LAM
Chương 4. MÔ PHỎNG
Chương 5. HAI CON TRỎ
Chương 6. CHẶT NHỊ PHÂN
Chương 7. KHỬ VÒNG LẶP
Chương 8. ĐỆ QUY, QUAY LUI VÀ VÉT CẠN
Chương 9. QUY HOẠCH ĐỘNG
Chương 10. ĐỒ THỊ CƠ BẢN
Chương 11. QUY HOẠCH ĐỘNG NÂNG CAO VÀ VẬN DỤNG
Phụ lục. CHUẨN BỊ CHO PHÒNG THI

Dành cho học sinh THPT ôn luyện HSG tỉnh

LỜI NÓI ĐẦU

Hầu hết các bài thi học sinh giỏi Tin học cấp tỉnh đều có chung một "kịch bản": Viết một chương trình chạy đúng thì không khó, nhưng viết một chương trình vừa đúng vừa chạy kịp trong 1 giây với dữ liệu lớn thì khó hơn rất nhiều. Cách giải đầu tiên mà ai cũng nghĩ ra - thử hết mọi khả năng - được gọi vui là thuật toán trâu. Trâu khỏe, trâu chăm chỉ, nhưng trâu chậm. Tài liệu này dạy em cách đi từ con trâu ấy đến một lời giải nhanh, gọn và đúng.

Tài liệu gồm mười một chương, sắp xếp theo lộ trình học từ nền tảng đến nâng cao, bám sát thứ tự bốn câu của đề thi: Toán học và cách chống trâu (dùng công thức, cộng dồn, sàng, đếm thông minh, lũy thừa nhanh); Số học, xâu và sắp xếp (thuật toán Euclid, sắp xếp theo tiêu chí riêng, đếm chữ cái bằng cộng dồn); Tham lam (mỗi bước chọn cái tốt nhất trước mắt - và chứng minh vì sao được phép làm thế); Mô phỏng (làm đúng từng bước như đề nói - rồi tìm cách nhảy qua, gộp lại hoặc đếm thay cho những bước lặp lại); Hai con trỏ (hai người cùng đi một chiều, không ai phải quay lại); Chặt nhị phân (mỗi câu hỏi loại đi một nửa khả năng); Khử vòng lặp (nhìn bài toán bằng con mắt toán học để vòng lặp biến mất - đếm theo hàng chữ số, phân tích thừa số nguyên tố, gom nhóm theo số dư, tìm quy luật); Đệ quy, quay lui và vét cạn (thử hết mọi khả năng một cách có hệ thống và biết cắt bỏ ngõ cụt); Quy hoạch động (nhớ lại những gì đã tính để khỏi tính lại); Đồ thị cơ bản (DFS, thành phần liên thông, BFS, Dijkstra); Quy hoạch động nâng cao và vận dụng (QHĐ trên đoạn, QHĐ trạng thái tập con, phối hợp nhiều kĩ thuật). Phụ lục hướng dẫn đọc ghi tệp, cách chấm bằng Themis, tự kiểm tra lời giải và lưu ý khi lập trình bằng Python.

Chương 1 - 3 phục vụ trước hết Câu 1 và Câu 2 của đề (rẽ nhánh, vòng lặp, mảng, xâu, sắp xếp, số học, tham lam đơn giản); Chương 4 - 8 là các kĩ thuật của Câu 2 và Câu 3 (mô phỏng, hai con trỏ, chặt nhị phân, khử vòng lặp, vét cạn, quay lui); Chương 9 - 11 dành cho Câu 3 và Câu 4 (quy hoạch động, đồ thị, vận dụng cao) theo Cấu trúc đề thi chọn học sinh giỏi cấp tỉnh lớp 12 môn Tin học của Sở GD&ĐT Phú Thọ (năm học 2025 - 2026). Bảy chương Toán học, Tham lam, Mô phỏng, Hai con trỏ, Chặt nhị phân, Khử vòng lặp và Quy hoạch động chính là "bảy con đường tư duy" làm nên tên của cuốn sách; bốn chương còn lại bổ sung phần kiến thức nền tảng và nâng cao mà đề thi yêu cầu. Nên học theo đúng thứ tự các chương: Lời giải chính của mỗi bài chỉ dùng kiến thức đã học ở các bài trước nó; những chỗ nhắc tới một bài phía sau chỉ là phần mở rộng hoặc bài luyện thêm, em có thể quay lại sau.

Mỗi bài học không đưa ra kiến thức ngay. Em sẽ được đặt trước một bài toán cụ thể, tự tay làm thử, viết thuật toán ngây thơ, chạy tay từng bước trên hình vẽ, rồi tự cảm nhận chỗ chậm. Từ chỗ chậm đó, sách dẫn em đi từng bước nhỏ - mỗi bước một quan sát, một hình vẽ - cho đến khi ý tưởng tốt hiện ra. Sau đó em sẽ thấy vì sao ý tưởng đó đúng, nó nhanh đến đâu, và có những cái bẫy nào cần tránh.

Lê Thành Nam

Các ví dụ trong sách được chọn lọc từ những dạng bài đã xuất hiện trong các đề khảo sát đội tuyển và từ bộ sách "Tư duy thuật toán qua Toán học". Phần **Liên hệ đề thi** ở cuối mỗi bài chỉ ra em sẽ gặp lại kĩ thuật ấy ở đâu. Phần **Luyện tập** được xếp theo ba mức sao: ★ làm ngay được sau khi đọc bài, ★★ cần suy nghĩ thêm một bước, ★★★ dành cho em muốn thử sức.

Code trong sách viết bằng C++ và chỉ trình bày phần ý chính (không có phần đọc, ghi tệp đầy đủ) để em tập trung vào thuật toán. Khi đã hiểu, em hãy tự viết lại chương trình hoàn chỉnh - đó là cách học tốt nhất.

Chúc các em học tập say mê và thi đạt kết quả cao!

Lê Thành Nam

CÁCH HỌC VỚI TÀI LIỆU NÀY

Mỗi bài học được xây dựng theo cùng một con đường, giống như cách một người giỏi thực sự suy nghĩ khi gặp bài mới:

1. Khởi động - Một câu hỏi nhỏ, gần gũi để em bắt đầu nghĩ.

2. Bài toán - Đề bài cụ thể có ví dụ.

3. Ý tưởng ngây thơ - Cách làm đầu tiên ai cũng nghĩ ra, được chạy tay từng bước.

4. Bế tắc - Đo xem cách ngây thơ chậm đến mức nào.

5. Phá bế tắc - Từng quan sát nhỏ dẫn đến ý tưởng tốt.

6. Thuật toán tốt - Chạy tay trên hình và code ý chính.

7. Vì sao đúng, độ phức tạp, bẫy thường gặp.

8. Liên hệ đề thi và luyện tập.

Đừng đọc lướt! Khi gặp hình chạy tay, hãy lấy giấy nháp làm lại trước khi nhìn hình. Khi gặp khung "Em hãy suy nghĩ", hãy dừng lại ít nhất vài phút. Các khung màu trong sách có ý nghĩa như sau:

Ý TƯỞNG NGÂY THƠ cách làm đầu tiên đơn giản nhất - thường đúng nhưng chậm.

BẾ TẮC đo xem cách ngây thơ tại chậm đến đâu, vì sao không đủ.

PHÁ BẾ TẮC từng bước quan sát dẫn đến ý tưởng tốt hơn.

VÌ SAO ĐÚNG? lập luận chứng minh thuật toán luôn cho kết quả đúng.

ĐỘ PHỨC TẠP đếm số phép tính, so sánh với giới hạn thời gian.

GHI NHỚ điều cốt lõi cần nhớ sau bài học.

BẪY THƯỜNG GẶP những lỗi học sinh hay mắc khi cài đặt.

LIÊN HỆ ĐỀ THI kĩ thuật này xuất hiện ở những đề thi nào.

Lê Thành Nam

MỤC LỤC

CHƯƠNG 1. TOÁN HỌC VÀ CÁCH CHỐNG TRÂU 7

Bài 1. Đếm phép tính - thước đo của thuật toán 8
Bài 2. Mảng cộng dồn - trả lời nghìn câu hỏi trong chớp mắt 17
Bài 3. Ước số và số nguyên tố - chỉ cần đi đến căn bậc hai 31
Bài 4. Đếm mà không cần liệt kê 42
Bài 5. Lũy thừa nhanh và phép chia lấy dư 52

CHƯƠNG 2. SỐ HỌC, XÂU VÀ SẮP XẾP 60

Bài 6. Thuật toán Euclid - ƯCLN và BCNN trong chớp mắt 61
Bài 7. Sắp xếp theo tiêu chí riêng - em đặt luật, máy sắp xếp 71
Bài 8. Xâu kí tự - đếm chữ cái và cộng dồn theo từng chữ 80

CHƯƠNG 3. THAM LAM 88

Bài 9. Đổi tiền - tham lam là gì? 89
Bài 10. Chọn lịch họp - chọn đúng tiêu chí tham lam 95
Bài 11. Xếp hàng chờ - so sánh hai người đứng cạnh nhau 104
Bài 12. Nối dây - tham lam với hàng đợi ưu tiên 111

CHƯƠNG 4. MÔ PHỎNG 117

Bài 13. Mô phỏng trực tiếp - làm đúng từng bước như đề nói 118
Bài 14. Mô phỏng có chu kì - nhảy qua những vòng lặp lại 127
Bài 15. Mô phỏng theo sự kiện - chỉ dừng lại khi có chuyện xảy ra 134
Bài 16. Mô phỏng thông minh - đếm thay vì chạy 139

CHƯƠNG 5. HAI CON TRỎ 145

Bài 17. Hai con trỏ đi ngược chiều - tìm cặp có tổng X 146
Bài 18. Cửa sổ trượt - hai con trỏ cùng chiều 154
Bài 19. Đếm đoạn con bằng hai con trỏ 162
Bài 20. Hai con trỏ trên hai dãy 168

CHƯƠNG 6. CHẶT NHỊ PHÂN 178

Bài 21. Tìm kiếm nhị phân - mỗi câu hỏi loại một nửa 179
Bài 22. Chặt nhị phân đáp án 186
Bài 23. Giá trị thứ K và chặt nhị phân trên số thực 193

CHƯƠNG 7. KHỬ VÒNG LẶP 200

Bài 24. Đếm chữ số theo từng hàng - không cần viết ra từng số 201
Bài 25. Số 0 tận cùng của n! trong hệ cơ số b - phân tích thay vì tính 208
Bài 26. Gom nhóm theo số dư - đếm cặp không cần ghép cặp 214
Bài 27. Tìm quy luật từ bảng nhỏ - trò chơi loại bỏ xen kẽ 220

CHƯƠNG 8. ĐỆ QUY, QUAY LUI VÀ VÉT CẠN 226

Bài 28. Duyệt mọi tập con - mỗi tập con là một số nhị phân 227
Bài 29. Quay lui - sinh hoán vị và tổ hợp 234
Bài 30. Cắt nhánh - đừng đi vào ngõ cụt 242

Lê Thành Nam

CHƯƠNG 9. QUY HOẠCH ĐỘNG 249
Bài 31. Leo cầu thang - quy hoạch động là gì? 250
Bài 32. Đường đi trên lưới - QHĐ hai chiều 258
Bài 33. Cái túi - chọn hay không chọn 266
Bài 34. Dãy con tăng dài nhất 274

CHƯƠNG 10. ĐỒ THỊ CƠ BẢN 281
Bài 35. Đồ thị và DFS - đếm các vùng liên thông 282
Bài 36. BFS - loang theo từng lớp, tìm đường ít bước nhất 291
Bài 37. Dijkstra - đường ngắn nhất khi cạnh dài khác nhau 298

CHƯƠNG 11. QUY HOẠCH ĐỘNG NÂNG CAO VÀ VẬN DỤNG 305
Bài 38. QHĐ trên đoạn - ghép dần từ đoạn ngắn đến đoạn dài 306
Bài 39. QHĐ trạng thái tập con - nhớ "đã dùng những gì" 312
Bài 40. Vận dụng tổng hợp - sắp xếp, cộng dồn, chặt nhị phân 318

PHỤ LỤC. CHUẨN BỊ CHO PHÒNG THI 325
Phụ lục A. Đọc và ghi tệp - đúng tên, đúng định dạng 326
Phụ lục B. Chấm bài bằng Themis - hiểu luật để lấy điểm 328
Phụ lục C. Tự kiểm tra lời giải - sinh test và đối chiếu 330
Phụ lục D. Lập trình bằng Python trong phòng thi 332
Phụ lục E. Bảng tra cứu nhanh - đọc giới hạn, chọn kĩ thuật 333

Lê Thành Nam

CHƯƠNG 1. TOÁN HỌC VÀ CÁCH CHỐNG TRÂU

Thuật toán trâu thử hết mọi khả năng. Nó luôn là điểm xuất phát tốt - nhưng hiếm khi là điểm kết thúc. Trong chương này, em sẽ học cách "đếm" xem một chương trình chậm đến đâu, rồi dùng những công cụ toán học quen thuộc - công thức tính tổng, cột mốc cộng dồn, cặp ước số, phép đếm thông minh, phép chia lấy dư - để biến những chương trình chạy hàng giờ thành những chương trình chạy trong chớp mắt.

TRONG CHƯƠNG NÀY

Bài 1. Đếm phép tính - thước đo của thuật toán
Bài 2. Mảng cộng dồn - trả lời nghìn câu hỏi trong chớp mắt
Bài 3. Ước số và số nguyên tố - chỉ cần đi đến căn bậc hai
Bài 4. Đếm mà không cần liệt kê
Bài 5. Lũy thừa nhanh và phép chia lấy dư

Lê Thành Nam

BÀI 1 Đếm phép tính - thước đo của thuật toán

Học xong bài này, em sẽ:

✓ Biết đếm xem một chương trình phải làm khoảng bao nhiêu phép tính.

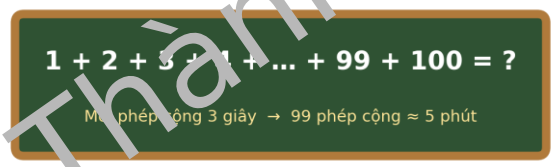
✓ Hiểu kí hiệu độ phức tạp O(1), O(n), O(n²), O(log n)… và dùng nó để dự đoán chương trình có chạy kịp 1 giây hay không.

✓ Trải nghiệm lần đầu cách "chống trâu": Thay một vòng lặp dài bằng một công thức toán học.

KHỞI ĐỘNG

Năm 1787, thầy giáo của cậu bé Gauss (10 tuổi) giao cho cả lớp bài toán: Cộng tất cả các số từ 1 đến 100. Thầy nghĩ cả lớp sẽ ngồi im ít nhất nửa giờ.

Nếu mỗi phép cộng mất 3 giây, em sẽ mất bao lâu? (Có 99 phép cộng, khoảng 5 phút.) Thế nhưng chỉ vài giây sau, Gauss đã nộp bảng với đáp số 5050. Cậu không cộng nhanh hơn các bạn - cậu cộng ít hơn. Bài học này sẽ cho em thấy cậu đã làm thế nào, và vì sao đó chính là tinh thần của mọi thuật toán tốt.



Hình 1.1. Bài toán của thầy giáo cậu bé Gauss

BÀI TOÁN: TỔNG [TONG]

Cho số nguyên dương n. Hãy tính $S = 1 + 2 + 3 + \ldots + n$.

Dữ liệu vào: Một số nguyên n ($1 \le n \le 10^9$).

Kết quả: Ghi ra giá trị S.

Dữ liệu vào	Kết quả	Giải thích
6	21	1 + 2 + 3 + 4 + 5 + 6 = 21
100	5050	Bài toán của cậu bé Gauss

1. Ý tưởng ngây thơ: Cộng lần lượt

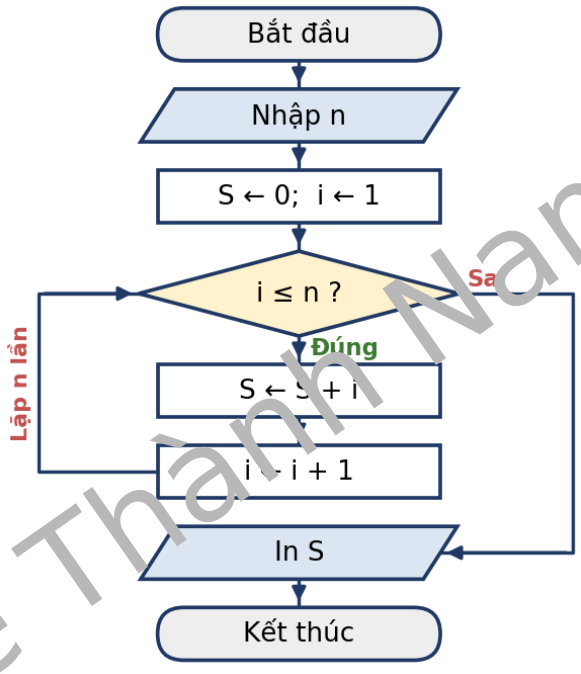
Cách đầu tiên ai cũng nghĩ ra là bắt chước đúng điều đề bài nói: Lấy một "cái túi" S ban đầu bằng 0, rồi lần lượt bỏ vào túi các số 1, 2, 3, … cho đến n.

Lê Thành Nam

Ý TƯỞNG NGÂY THƠ - CỘNG DẦN

- Khởi tạo S = 0.
- Cho i chạy từ 1 đến n: Cộng thêm i vào S.
- Ghi ra S.

Sơ đồ khối của thuật toán ngây thơ:



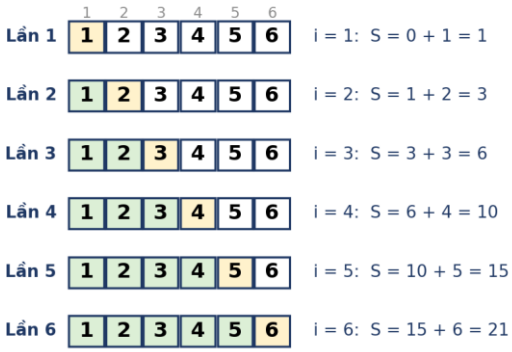
Hình 1.2. Sơ đồ khối thuật toán cộng dần

Lê Thành Nam

VÌ SAO CÁCH NGÂY THƠ CHẬM? - ĐẾM TỪNG BƯỚC

- Hai bước khởi tạo (S ← 0, i ← 1) và bước in kết quả: Mỗi bước làm đúng **1 lần** - không phụ thuộc n.
- Vòng lặp (khung đỏ "lặp n lần"): Biến i nhận lần lượt các giá trị 1, 2, …, n, nên thân vòng lặp chạy đúng **n lần**. Mỗi lần làm 3 việc: So sánh i ≤ n, cộng S ← S + i, tăng i. Tổng 3n phép tính.
- Cộng lại: Khoảng 3n + 3 phép tính. Khi n lớn, hằng số 3 ở trước và số 3 cộng thêm không làm thay đổi "kiểu tăng": n gấp đôi thì số phép tính cũng gấp đôi. Ta viết gọn là **O(n)** - đọc là "cỡ n".

Hãy chạy tay với n = 6. Mỗi hàng của hình dưới là một lần lặp: Ô vàng là số vừa được cộng, các ô xanh là những số đã nằm trong túi.



***Hình 1.3.** Chạy tay thuật toán cộng dần với n = 6: Sau 6 lần lặp, S = 21*

Code ý chính - thuật toán cộng dần

```
long long sum = 0;
for (long long i = 1; i <= n; i++)
    sum += i;
cout << sum;
```

Chương trình này **đúng** với mọi n. Nhưng nó có **nhanh** không? Muốn trả lời, ta phải đếm xem máy làm bao nhiêu việc.

2. Đếm phép tính

Máy tính không "tính" ra đáp số như người. Nó làm từng phép tính nhỏ: Một phép cộng, một phép so sánh, một lần gán. Một máy tính thông thường làm được khoảng **10^8 (một trăm triệu) phép tính đơn giản mỗi giây** - con số này chỉ là ước lượng, nhưng đủ dùng để dự đoán.

Vòng lặp trên chạy đúng n lần, mỗi lần làm một phép cộng, một phép so sánh $i \le n$ và một phép tăng i. Ta nói thuật toán làm **khoảng n phép tính** (không cần quan tâm đến hệ số 3).

n	Số lần lặp	Thời gian ước lượng
10^3	1 000	Tức thì
10^6	1 000 000	≈ 0,01 giây
10^8	100 000 000	≈ 1 giây
10^9	1 000 000 000	≈ 10 giây - QUÁ GIỜ

Lê Thành Nam

BẾ TẮC

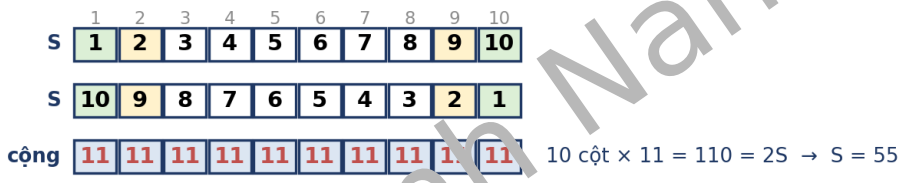
Đề bài cho n tới 10^9, mà giới hạn thời gian thường chỉ là 1 giây. Thuật toán cộng dần cần khoảng 10^9 phép tính - chậm gấp chừng 10 lần mức cho phép.

Có cố viết "gọn hơn" cũng vô ích: Dù viết thế nào, nếu vẫn cộng từng số một thì vẫn phải cộng n lần. Muốn nhanh hơn, ta phải **cộng ít hơn** - giống Gauss.

3. Phá bế tắc: Cách nhìn của Gauss

Bước 1. Viết tổng hai lần - một lần xuôi, một lần ngược

Lấy n = 10 cho dễ nhìn. Viết dãy 1, 2, …, 10 ở hàng trên; ngay dưới, viết lại chính dãy đó nhưng theo thứ tự ngược 10, 9, …, 1. Cả hai hàng đều có tổng bằng S.



Hình 1.4. Cộng theo cột: Cặp đầu - cuối (xanh), cặp thứ hai (vàng)… cột nào cũng bằng 11

Em thấy gì? Ở mỗi cột, số trên tăng 1 thì số dưới giảm 1, nên **tổng mỗi cột không đổi**: 1 + 10 = 2 + 9 = 3 + 8 = … = 11.

Bước 2. Đếm số cột thay vì cộng từng số

Có 10 cột, mỗi cột bằng 11, nên hai hàng cộng lại bằng 10 × 11 = 110. Nhưng hai hàng chính là S + S = 2S. Vậy 2S = 110, tức là S = 55.

Bước 3. Tổng quát cho n bất kì

Với n tùy ý: Hàng trên là 1, 2, …, n; hàng dưới là n, n − 1, …, 1. Cột thứ k có tổng k + (n + 1 − k) = n + 1. Có n cột, nên 2S = n·(n + 1).

CÔNG THỨC GAUSS

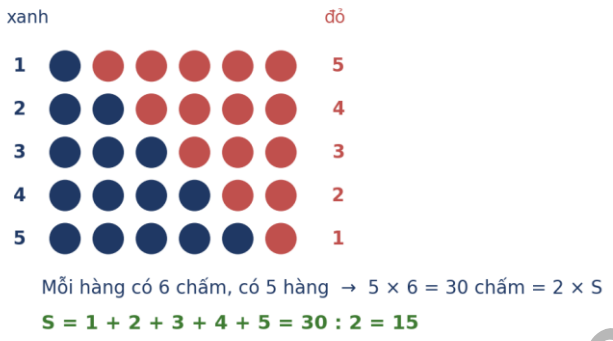
S = 1 + 2 + … + n = n·(n + 1) / 2.

Công thức chỉ cần 1 phép nhân và 1 phép chia - dù n là 10 hay 10^9.

Lê Thành Nam

Bước 4. Nhìn bằng hình để chắc chắn

Nếu em chưa thật tin, hãy xếp các chấm: Hàng thứ k có k chấm xanh (đó là S). Ghép thêm một "cầu thang" đỏ giống hệt nhưng lật ngược. Hai cầu thang khớp thành một hình chữ nhật.



Hình 1.5. *Hai "cầu thang" 1 + 2 + … + 5 ghép thành hình chữ nhật 5 × 6*

Code ý chính - công thức Gauss

```
long long sum = n * (n + 1) / 2;
cout << sum;
```

Lê Thành Nam

VÌ SAO ĐÚNG?

Gọi S = 1 + 2 + … + n. Viết S = n + (n − 1) + … + 1. Cộng hai đẳng thức theo từng cặp số hạng cùng vị trí: Mỗi cặp có tổng n + 1, và có n cặp, nên 2S = n(n + 1).

Vì n và n + 1 là hai số liên tiếp, một trong hai số là số chẵn, nên n(n + 1) luôn chia hết cho 2 - phép chia không bao giờ bị lệch.

ĐỘ PHỨC TẠP

- **Cộng dần:** Vòng lặp chạy n lần, mỗi lần một số cố định phép tính ⇒ số phép tính tỉ lệ với n ⇒ **O(n)**.
- **Công thức Gauss:** Chỉ có 1 phép cộng (n + 1), 1 phép nhân, 1 phép chia - luôn 3 phép tính dù n = 10 hay n = 10^9. Số phép tính **không phụ thuộc n ⇒ O(1)** (hằng số).
- Với n = 10^9: Cách 1 cần khoảng $3 \cdot 10^9$ phép tính (vài chục giây), cách 2 cần 3 phép tính (chưa đến một phần triệu giây).

BẪY THƯỜNG GẶP

- Với $n = 10^9$ thì $n \cdot (n + 1) \approx 10^{18}$, vượt xa giới hạn của kiểu int (khoảng $2{,}1 \cdot 10^9$). Phải khai báo n là **long long** trước khi nhân, nếu không phép nhân sẽ bị tràn số và cho kết quả sai (thậm chí âm).
- Viết n * (n + 1) / 2, không viết n / 2 * (n + 1): Khi n lẻ, n / 2 bị làm tròn xuống và kết quả sai.

4. Độ phức tạp - ngôn ngữ chung để nói "nhanh" hay "chậm"

Từ ví dụ trên, ta rút ra cách đánh giá mọi thuật toán: **Đếm số phép tính theo kích thước dữ liệu n, bỏ qua hệ số, chỉ giữ phần lớn nhất**. Kết quả được viết trong dấu O( ) và gọi là **độ phức tạp thời gian**.

- Một vòng lặp chạy n lần → **O(n)**.
- Hai vòng lặp lồng nhau, mỗi vòng chạy n lần → n·n lần → **O(n^2)**.
- Vòng lặp mà mỗi lần biến chạy bị chia đôi (n, n/2, n/4, …, 1) → chạy khoảng $\log_2 n$ lần → **O(log n)**. Với $n = 10^9$ chỉ khoảng 30 lần!
- Thuật toán $3n^2 + 5n + 7$ phép tính và ta viết là $O(n^2)$: Khi n lớn, phần n^2 lấn át tất cả.

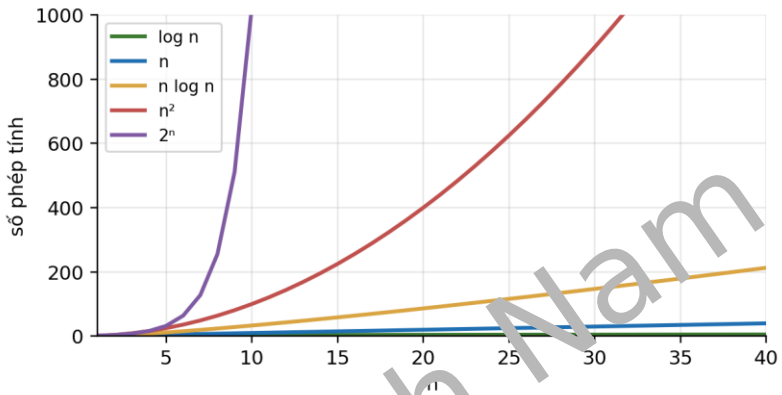
Để tự tìm độ phức tạp của một đoạn code bất kì, em chỉ cần năm quy tắc dưới đây. Mọi hộp "Độ phức tạp" trong sách đều dùng đúng các quy tắc này.

Quy tắc	Giải thích	Ví dụ
1. Lệnh đơn	Phép gán, cộng, so sánh, đọc một ô mảng: Tốn một lượng thời gian cố định	O(1)
2. Nối tiếp ⇒ cộng	Làm xong phần A rồi mới làm phần B: Cộng số phép tính, rồi giữ phần lớn nhất	$O(n) + O(n^2) = O(n^2)$
3. Lồng nhau ⇒ nhân	Vòng ngoài chạy a lần, mỗi lần vòng trong chạy b lần: Thân trong chạy a × b lần	$n \times n = O(n^2)$
4. Chia đôi ⇒ log	Mỗi bước kích thước còn một nửa: n → n/2 → … → 1 mất khoảng $\log_2 n$ bước	O(log n)
5. Bỏ hằng số	Bỏ hệ số nhân và các phần nhỏ hơn: $3n^2 + 5n + 7 \to n^2$	$O(n^2)$

Lê Thành Nam

> **CHÚ Ý**
>
> Quy tắc 3 cho **giới hạn trên**. Có khi vòng trong không chạy đủ b lần mỗi lượt - ví dụ con trỏ chỉ tiến (Chương 5): Khi đó phải đếm **tổng số bước** của vòng trong trong cả chương trình, thường nhỏ hơn nhiều so với a × b.

Hình dưới cho thấy các "loại tốc độ" tăng khác nhau đến mức nào khi n lớn dần.



Hình 1.6. Số phép tính tăng theo n của các độ phức tạp thường gặp

Đọc giới hạn n để đoán thuật toán cần làm

Đây là kĩ năng quan trọng nhất khi đọc đề thi. Từ giới hạn của n và 10^8 phép tính mỗi giây, ta suy ra độ phức tạp tối đa được phép:

Giới hạn n	Độ phức tạp chấp nhận	Thường gặp
$n \le 10$	$O(n!)$	Thử mọi hoán vị
$n \le 20$	$O(2^n \cdot n)$	Thử mọi tập con
$n \le 500$	$O(n^3)$	Ba vòng lặp lồng nhau
$n \le 5\,000$	$O(n^2)$	Xét mọi cặp, QHĐ hai chiều
$n \le 10^6$	$O(n \log n)$	Sắp xếp, chặt nhị phân
$n \le 10^8$	$O(n)$	Một lần duyệt, hai con trỏ, cộng dồn
$n \le 10^{18}$	$O(\log n)$ hoặc $O(1)$	Công thức, lũy thừa nhanh

Đếm số lần lặp của hai vòng lồng nhau

Rất nhiều thuật toán trâu có dạng "xét mọi cặp (i, j) với i < j". Đoạn code đó chạy bao nhiêu lần?

Lê Thành Nam

```
for (int i = 1; i <= n; i++)
    for (int j = i + 1; j <= n; j++)
        check(i, j);
```

Hãy vẽ bảng n × n: Hàng i, cột j. Vòng lặp chỉ đi qua các ô **nằm trên đường chéo** (j > i).



Hình 1.7. Các ô (i, j) với i < j khi n = 6: Có 15 ô

Hàng 1 có n − 1 ô, hàng 2 có n − 2 ô, ..., hàng n có 0 ô. Tổng là (n − 1) + (n − 2) + … + 1 + 0 - lại chính là công thức Gauss: **n(n − 1)/2** lần. Với n = 10^5, đó là khoảng $5 \cdot 10^9$ lần - quá chậm. Đây là lí do ta cần các chương sau.

Lê Thành Nam

GHI NHỚ

- Trước khi viết code, hãy **đếm** số phép tính và so với 10^8.
- Quy trình chống trâu: (1) viết thuật toán trâu đúng; (2) đếm xem nó chậm ở đâu; (3) tìm phần việc bị lặp lại hoặc thừa; (4) dùng toán học hoặc một kĩ thuật để bỏ phần thừa đó; (5) so kết quả thuật toán mới với thuật toán trâu trên nhiều test nhỏ.
- Thuật toán trâu không vô dụng: Nó là "trọng tài" để kiểm tra thuật toán tốt, và thường vẫn ăn được điểm subtask nhỏ.

LIÊN HỆ ĐỀ THI

Trong bộ 20 đề khảo sát đội tuyển (chuyên đề Hai con trỏ), **subtask 1 của mọi đề** đều dành cho thuật toán trâu $O(n^2)$ hoặc $O(n^3)$. Đọc giới hạn subtask theo bảng trên giúp em biết ngay thuật toán nào lấy được bao nhiêu điểm.

Bộ sách "Tư duy thuật toán qua Toán học" (Tập 1, chương Tính nhanh và tính nhẩm) có thêm nhiều tổng tính nhanh kiểu Gauss: Tổng số lẻ, tổng cấp số cộng, tổng "rút gọn kiểu kính viễn vọng".

LUYỆN TẬP

1.1. ★ Tính tổng các số chẵn $2 + 4 + \ldots + 2n$ bằng một công thức. Kiểm tra với $n = 5$ (đáp số 30).

1.2. ★ Tính tổng $a + (a + 1) + \ldots + b$ với $1 \le a \le b \le 10^9$ bằng $O(1)$. Gợi ý: Dùng hai lần công thức Gauss, hoặc "số cột × tổng mỗi cột".

1.3. ★ Đoạn code sau chạy bao nhiêu lần (theo n)? `for (int i = n; i > 0; i /= 2) cnt++;` Tính cụ thể với $n = 1000$.

1.4. ★★ Đếm số lần gọi check(i, j, k) trong ba vòng lặp lồng nhau với $1 \le i < j < k \le n$. Kiểm tra với $n = 5$ (đáp số 10).

1.5. ★★ Biết $1^2 + 2^2 + \ldots + n^2 = n(n + 1)(2n + 1)/6$. Viết cả chương trình trâu và chương trình dùng công thức, rồi tự sinh 100 giá trị n ngẫu nhiên nhỏ để so sánh kết quả hai chương trình.

1.6. ★★ Cho $n \le 10^9$. Đếm số cặp (i, j), $1 \le i < j \le n$, mà $i + j$ là số chẵn. (Gợi ý: Hai số cùng chẵn hoặc cùng lẻ.)

1.7. ★★★ Tính $(1 + 2 + \ldots + n) \bmod (10^9 + 7)$ với $n \le 10^{18}$. Chú ý: $n(n + 1)$ không còn vừa long long; hãy chia 2 cho thừa số chẵn trước, rồi lấy dư từng thừa số trước khi nhân.

Lê Thành Nam